

GenAI Application Development: Phase 4

Study Guide: Orchestration & Agentic Systems

1. Select LangChain/similar tools for use in a GenAI application

Source: <https://docs.databricks.com/en/machine-learning/langchain.html>

Core Concept: LangChain is an open-source orchestration framework that enables developers to build stateful applications by chaining together LLMs, vector stores, and external tools. It acts as the routing logic for generative AI apps.

How it works on Databricks: Databricks provides native integrations via `langchain-databricks`, providing components like `ChatDatabricks` (for Model Serving endpoints) and `DatabricksVectorSearch`. The MLflow LangChain flavor (`mlflow.langchain`) automatically serializes these chains and logs them to your MLflow tracking server.

When to use vs. alternatives:

- **Use LangChain (or LangGraph):** When your application requires multiple steps, tool-calling (e.g., retrieving from a database, then summarizing), memory management, or complex conditional routing.
- **Alternatives:** Direct API calls (e.g., raw OpenAI SDK). Use direct API calls only for simple, stateless, single-turn text generation tasks to avoid unnecessary overhead.

Common Mistakes: Failing to use MLflow's autologging feature when experimenting with LangChain. Without autologging, debugging complex multi-step chains is extremely difficult as you lose visibility into intermediate LLM inputs and outputs.

Code Example & Function: Enabling autologging and initializing a Databricks LLM in LangChain.

```
import mlflow
from langchain_databricks import ChatDatabricks

# Function: mlflow.langchain.autolog()
# Automatically records traces, inputs, outputs, and parameters of LangChain executions
# into MLflow for debugging and evaluation.
mlflow.langchain.autolog()

# Initialize the LangChain chat model pointing to a Databricks endpoint
llm = ChatDatabricks(endpoint="databricks-meta-llama-3-1-70b-instruct")

response = llm.invoke("What are agentic systems?")
print(response.content)
```

2. Utilize MLflow and Agent Framework for developing agentic systems

Source: <https://docs.databricks.com/en/generative-ai/agent-framework/index.html> & <https://docs.databricks.com/en/mlflow/models.html>

Core Concept: Databricks Mosaic AI Agent Framework is a set of tools integrated directly with MLflow and Unity Catalog designed to author, trace, evaluate, and deploy autonomous GenAI agents securely in production.

How it works on Databricks: You author an agent (using LangChain or custom Python/PyFunc). Instead of deploying raw code, you log the agent to MLflow. MLflow packages the model with its dependencies and input/output signatures. You then deploy this MLflow model to a Databricks Model Serving endpoint, which natively integrates with Databricks native tracing and Unity Catalog permissions for secure data access.

When to use vs. alternatives:

- **Use Mosaic AI Agent Framework:** When building enterprise-grade agents that require secure access to internal data, native performance tracing, and integrated LLM-as-a-judge evaluation (Mosaic AI Agent Evaluation).
- **Alternatives:** Hosting custom Docker containers on cloud services (e.g., AWS ECS). This lacks Databricks-native governance, integrated tracing, and unified billing.

Common Mistakes: Deploying agents without explicitly defining an MLflow model signature (input/output schema). Without a signature, Model Serving endpoints may fail to parse incoming REST API requests correctly, leading to 400 Bad Request errors.

Code Example & Function: Logging an agent logic block using MLflow.

```

import mlflow
from mlflow.models import infer_signature

# Function: mlflow.models.set_model()
# Logs the custom agent logic as an MLflow model within an active run,
# capturing the environment and defining the expected signature.
with mlflow.start_run():
    # Assume 'chain' is a pre-defined LangChain object
    signature = infer_signature(
        model_input={"messages": [{"role": "user", "content": "hello"}]},
        model_output="string_response"
    )

    mlflow.langchain.log_model(
        lc_model=chain,
        artifact_path="agent_model",
        signature=signature
    )

```

3. Enable multi-agent systems to leverage Genie Spaces or conversational API

Source: <https://docs.databricks.com/en/genie/index.html> & <https://docs.databricks.com/en/sql/user/genie/index.html>

Core Concept: Databricks AI/BI Genie provides a secure, governed semantic layer where business rules and relationships are pre-defined over structured data. Multi-agent systems can route data queries to Genie rather than attempting to write raw, unstructured SQL.

How it works on Databricks: *(Note: While Databricks UI allows direct conversation with Genie, public documentation on a dedicated "Genie Conversational API" specifically for programmatic agent routing is evolving).* Generally, to retrieve data programmatically, developers wrap Databricks SQL execution commands (or use LangChain's Databricks SQL Toolkit) as tools. The agent orchestrator determines if a question requires querying a data warehouse, formulates the natural language or semantic request, and executes it against the Databricks SQL endpoint or defined Genie semantic layer, returning trusted tables to the agent.

When to use vs. alternatives:

- **Use Genie / Semantic APIs:** When agents need to query enterprise databases where precise business logic (e.g., "how is net revenue calculated?") is strictly governed.
- **Alternatives:** Text-to-SQL generation. Standard LLM Text-to-SQL is prone to hallucinations and joins tables incorrectly if not heavily prompted, making semantic layers significantly safer for agents.

Common Mistakes: Giving an agent raw, unrestricted SQL execution access across all Unity Catalog tables rather than scoping its access to a curated semantic model or specific, read-only views, risking data privacy breaches or massive compute costs.

Code Example & Function: Providing an agent access to Databricks data via a tool (using LangChain Databricks SQL toolkit).

```
from langchain_community.utilities.databricks_sql import DatabricksSQLDatabase
from langchain_community.agent_toolkits import SQLDatabaseToolkit

# Function: DatabricksSQLDatabase.from_databricks()
# Connects the LangChain tool securely to a Databricks SQL Warehouse,
# allowing the agent to query data conversationally.
db = DatabricksSQLDatabase.from_databricks(
    catalog="main",
    schema="semantic_layer"
)

toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

# The agent can now use these 'tools' to answer data-related questions securely.
```